

SYSTEM-DESIGN

Computer Architecture (Disk Storage, RAM, Cache, CPU)

Tutorial thiết kế hệ thống hoàn chỉnh này bao quát các chủ đề như khả năng mở rộng (scalability), độ tin cậy (reliability), xử lý dữ liệu và kiến trúc tổng thể ở mức cao, với những giải thích rõ ràng, ví dụ thực tế và các chiến lược mang tính thực hành. Khóa học này sẽ dạy bạn những khái niệm cốt lõi mà bạn cần biết cho một buổi phỏng vấn thiết kế hệ thống.

Đây là một khóa học cấp tốc đầy đủ về các khái niệm thiết kế hệ thống mà bạn cần nắm vững để vượt qua vòng phỏng vấn xin việc. Phỏng vấn thiết kế hệ thống không tập trung nhiều vào việc viết code, và người phỏng vấn cũng không muốn thấy bạn viết mã nguồn chi tiết, mà họ quan tâm đến cách bạn kết nối và tổ chức toàn bộ hệ thống với nhau. Và đó chính xác là những gì chúng ta sẽ đề cập trong tutorial này.

Chúng ta sẽ đi qua tất cả các khái niệm mà bạn cần biết trước khi tham gia phỏng vấn thiết kế hệ thống. Trước khi bắt đầu thiết kế các hệ thống phân tán quy mô lớn, điều quan trọng là phải hiểu kiến trúc tổng thể ở mức cao của một máy tính đơn lẻ.

Hãy cùng xem các thành phần khác nhau của máy tính phối hợp với nhau như thế nào để thực thi mã nguồn của chúng ta. Máy tính hoạt động dựa trên một hệ thống phân tầng, trong đó mỗi tầng được tối ưu cho những nhiệm vụ khác nhau.

Ở tầng thấp nhất, máy tính chỉ hiểu các giá trị nhị phân là số 0 và số 1. Những giá trị này được biểu diễn dưới dạng bit. Một bit là đơn vị dữ liệu nhỏ nhất trong máy tính và chỉ có thể là 0 hoặc 1. Một byte gồm 8 bit và được dùng để biểu diễn một ký tự đơn lẻ như chữ “a” hoặc một chữ số như “1”.

Mở rộng từ đó, chúng ta có các đơn vị lưu trữ lớn hơn như kilobyte, megabyte, gigabyte và terabyte. Để lưu trữ dữ liệu này, chúng ta sử dụng bộ nhớ đĩa của máy tính, nơi chứa dữ liệu chính. Đĩa có thể là loại HDD hoặc SSD.

Bộ nhớ đĩa là bộ nhớ không bay hơi (non-volatile), nghĩa là nó vẫn giữ dữ liệu ngay cả khi không có nguồn điện. Vì vậy, khi bạn tắt hoặc khởi động lại máy tính, dữ liệu vẫn còn nguyên. Đĩa chứa hệ điều hành, các ứng dụng và toàn bộ tệp của người dùng.

Về dung lượng, đĩa thường có kích thước từ vài trăm gigabyte đến nhiều terabyte. Mặc dù SSD đắt hơn, chúng cung cấp tốc độ truy xuất dữ liệu nhanh hơn rất nhiều so với HDD. Ví dụ, một SSD có thể có tốc độ đọc từ 500 MB mỗi giây đến 3.500 MB mỗi giây, trong khi HDD thường chỉ đạt khoảng 80 đến 160 MB mỗi giây.

Điểm truy cập dữ liệu tiếp theo ngay sau đĩa là RAM (Random Access Memory – bộ nhớ truy cập ngẫu nhiên). RAM đóng vai trò là nơi lưu trữ dữ liệu đang hoạt động chính,

bao gồm các cấu trúc dữ liệu, biến và dữ liệu của các ứng dụng đang được sử dụng hoặc xử lý.

Khi một chương trình chạy, các biến, kết quả tính toán trung gian, ngăn xếp runtime và nhiều thứ khác đều được lưu trong RAM vì nó cho phép đọc và ghi rất nhanh. Đây là bộ nhớ bay hơi (volatile), nghĩa là nó cần nguồn điện để giữ dữ liệu. Sau khi khởi động lại máy tính, dữ liệu trong RAM có thể sẽ bị mất.

Về dung lượng, RAM thường dao động từ vài gigabyte trên các thiết bị tiêu dùng đến hàng trăm gigabyte trên các máy chủ cao cấp. Tốc độ đọc ghi của RAM thường vượt quá 5.000 MB mỗi giây, nhanh hơn cả những SSD nhanh nhất.

Tuy nhiên, đôi khi ngay cả tốc độ này cũng chưa đủ, và đó là lý do bộ nhớ cache xuất hiện. Cache nhỏ hơn RAM rất nhiều, thường chỉ được đo bằng megabyte, nhưng thời gian truy cập của cache còn nhanh hơn RAM, chỉ mất vài nanosecond đối với cache L1.

CPU sẽ kiểm tra cache L1 đầu tiên để tìm dữ liệu. Nếu không tìm thấy, nó sẽ tiếp tục kiểm tra cache L2 và L3, và cuối cùng mới truy cập RAM. Mục đích của cache là giảm thời gian truy cập dữ liệu trung bình, vì vậy các dữ liệu được sử dụng thường xuyên sẽ được lưu ở đây để tối ưu hiệu suất của CPU.

Vậy còn CPU thì sao? CPU chính là bộ não của máy tính. Nó đảm nhiệm việc lấy lệnh, giải mã và thực thi các lệnh đó. Khi bạn chạy code, chính CPU là thành phần xử lý các thao tác được định nghĩa trong chương trình.

Tuy nhiên, trước khi CPU có thể chạy code của chúng ta – vốn được viết bằng các ngôn ngữ bậc cao như Java, C++, Python hoặc các ngôn ngữ khác – mã nguồn cần được biên dịch sang mã máy. Trình biên dịch (compiler) sẽ thực hiện việc chuyển đổi này. Khi mã đã được biên dịch thành mã máy, CPU mới có thể thực thi nó.

CPU có thể đọc và ghi dữ liệu từ RAM, đĩa và cache. Cuối cùng, chúng ta có bo mạch chủ (motherboard hay mainboard), đây là thành phần kết nối tất cả các bộ phận lại với nhau. Nó cung cấp các đường dẫn và bus cho phép dữ liệu di chuyển giữa các thành phần khác nhau trong hệ thống.

Production App Architecture (CI/CD, Load Balancers, Logging & Monitoring)

Bây giờ, hãy cùng xem kiến trúc tổng thể ở mức rất cao của một ứng dụng sẵn sàng cho môi trường production. Khu vực trọng tâm đầu tiên của chúng ta là pipeline CI/CD – Continuous Integration và Continuous Deployment.

CI/CD đảm bảo rằng mã nguồn của chúng ta được đưa từ repository, trải qua một loạt các bước kiểm tra, test và kiểm soát trong pipeline, rồi được triển khai lên server production hoàn toàn tự động, không cần bất kỳ sự can thiệp thủ công nào.

Pipeline này thường được cấu hình bằng các nền tảng như Jenkins hoặc GitHub Actions để tự động hóa toàn bộ quy trình triển khai. Khi ứng dụng đã chạy trên production, nó phải có khả năng xử lý một lượng lớn các request từ người dùng.

Việc này được quản lý bởi các load balancer và reverse proxy như NGINX. Chúng đảm bảo rằng các request của người dùng được phân phối đồng đều trên nhiều server khác nhau, giúp duy trì trải nghiệm mượt mà ngay cả khi lưu lượng truy cập tăng đột biến.

Server của chúng ta cũng cần lưu trữ dữ liệu. Vì vậy, chúng ta thường có một server lưu trữ bên ngoài, không chạy chung trên server production mà được kết nối thông qua mạng. Ngoài ra, các server còn có thể giao tiếp với nhiều server khác, tạo thành nhiều service khác nhau chứ không chỉ một service duy nhất.

Để đảm bảo mọi thứ vận hành trơn tru, chúng ta cần các hệ thống logging và monitoring, liên tục theo dõi từng tương tác nhỏ trong hệ thống, từ việc ghi log cho đến phân tích dữ liệu.

Thông lệ phổ biến là lưu trữ log trên các dịch vụ bên ngoài, thường nằm ngoài server production chính. Ở phía backend, các công cụ như PM2 có thể được dùng cho logging và monitoring. Ở phía frontend, các nền tảng như Sentry giúp ghi nhận và báo cáo lỗi theo thời gian thực.

Khi mọi thứ không diễn ra như mong đợi – tức là hệ thống logging phát hiện các request bị lỗi hoặc những hành vi bất thường – bước đầu tiên là kích hoạt hệ thống cảnh báo (alerting service).

Sau đó, các thông báo sẽ được gửi đi để thông tin cho người dùng, từ những thông báo chung chung như “có lỗi xảy ra” cho đến các thông báo cụ thể như “thanh toán thất bại”. Trong thực tế hiện đại, các cảnh báo này thường được tích hợp trực tiếp vào những nền tảng mà đội ngũ thường xuyên sử dụng, chẳng hạn như Slack.

Hãy tưởng tượng một channel Slack riêng biệt, nơi các cảnh báo xuất hiện ngay tại thời điểm sự cố xảy ra. Điều này cho phép các lập trình viên phản ứng gần như ngay lập tức, xử lý nguyên nhân gốc rễ của vấn đề trước khi nó leo thang thành sự cố nghiêm trọng hơn.

Sau đó, các lập trình viên cần tiến hành debug hệ thống. Trước hết, vấn đề phải được xác định rõ ràng. Những log mà chúng ta đã đề cập trước đó chính là điểm xuất phát đầu

tiên. Developers sẽ duyệt qua log để tìm các mẫu bất thường hoặc dấu hiệu có thể chỉ ra nguồn gốc của sự cố.

Tiếp theo, lỗi cần được tái hiện trong một môi trường an toàn. Nguyên tắc vàng là không bao giờ debug trực tiếp trên môi trường production. Thay vào đó, các lập trình viên sẽ tái tạo lỗi trên môi trường staging hoặc test, nhằm đảm bảo người dùng không bị ảnh hưởng trong quá trình debug.

Sau đó, các developer sử dụng các công cụ để “nhìn vào bên trong” ứng dụng đang chạy và bắt đầu quá trình gỡ lỗi. Khi lỗi đã được khắc phục, một hotfix sẽ được triển khai.

Hotfix là một bản vá nhanh, mang tính tạm thời, được thiết kế để hệ thống hoạt động trở lại ngay lập tức. Nó giống như một miếng vá tạm trước khi một giải pháp lâu dài và hoàn chỉnh hơn được áp dụng.

Design Requirements (CAP Theorem, Throughput, Latency, SLOs and SLAs)

Trong phần này, chúng ta sẽ cùng tìm hiểu các trụ cột của thiết kế hệ thống và điều gì thực sự cần thiết để xây dựng một ứng dụng vững chắc và có khả năng chịu lỗi cao. Trước khi đi sâu vào các khía cạnh kỹ thuật, hãy nói về điều gì tạo nên một thiết kế tốt.

Khi nói đến thiết kế tốt trong kiến trúc hệ thống, chúng ta thực sự đang tập trung vào một vài nguyên tắc cốt lõi. Đầu tiên là khả năng mở rộng (scalability), tức là hệ thống có thể phát triển cùng với số lượng người dùng. Thứ hai là khả năng bảo trì (maintainability), đảm bảo rằng các lập trình viên trong tương lai có thể hiểu, chỉnh sửa và cải tiến hệ thống. Thứ ba là hiệu quả (efficiency), tức là tận dụng tài nguyên một cách tối ưu nhất.

Tuy nhiên, thiết kế tốt không chỉ dừng lại ở đó. Nó còn bao gồm việc lên kế hoạch cho các tình huống thất bại, xây dựng một hệ thống không chỉ hoạt động tốt khi mọi thứ diễn ra suôn sẻ mà còn giữ được sự ổn định khi có sự cố xảy ra.

Ở trung tâm của thiết kế hệ thống có ba yếu tố chính: di chuyển dữ liệu, lưu trữ dữ liệu và biến đổi dữ liệu.

Di chuyển dữ liệu là việc đảm bảo dữ liệu có thể luân chuyển trơn tru từ phần này sang phần khác của hệ thống, dù đó là request từ người dùng gửi đến server hay dữ liệu được truyền giữa các cơ sở dữ liệu. Ở đây, chúng ta cần tối ưu cả tốc độ lẫn bảo mật.

Lưu trữ dữ liệu không chỉ đơn thuần là chọn giữa cơ sở dữ liệu SQL hay NoSQL. Nó còn bao gồm việc hiểu các mô hình truy cập dữ liệu, chiến lược indexing và các giải pháp sao lưu. Chúng ta phải đảm bảo dữ liệu không chỉ được lưu trữ an toàn mà còn có thể truy cập nhanh chóng khi cần.

Biến đổi dữ liệu là quá trình lấy dữ liệu thô và chuyển nó thành thông tin có ý nghĩa. Ví dụ như tổng hợp các file log để phân tích, hoặc chuyển đổi dữ liệu đầu vào của người dùng sang một định dạng khác phù hợp với hệ thống.

Bây giờ, hãy dành chút thời gian để tìm hiểu một khái niệm cực kỳ quan trọng trong thiết kế hệ thống: Định lý CAP, còn được gọi là định lý Brewer, được đặt theo tên của nhà khoa học máy tính Eric Brewer.

Định lý này là tập hợp các nguyên tắc giúp chúng ta đưa ra những sự đánh đổi (trade-off) có cơ sở giữa ba thành phần quan trọng của một hệ thống phân tán: Consistency (tính nhất quán), Availability (tính sẵn sàng) và Partition Tolerance (khả năng chịu phân vùng).

Tính nhất quán đảm bảo rằng tất cả các node trong hệ thống phân tán đều có cùng một dữ liệu tại cùng một thời điểm. Nếu bạn thay đổi dữ liệu ở một node, sự thay đổi đó cũng phải được phản ánh trên tất cả các node khác. Hãy tưởng tượng việc chỉnh sửa một tài liệu Google Docs: khi một người chỉnh sửa, tất cả những người khác đều thấy thay đổi đó ngay lập tức.

Tính sẵn sàng có nghĩa là hệ thống luôn hoạt động và phản hồi các request, bất kể phía sau đang xảy ra điều gì. Giống như một cửa hàng trực tuyến đáng tin cậy, dù bạn truy cập lúc nào thì nó vẫn luôn mở cửa và sẵn sàng nhận đơn hàng.

Khả năng chịu phân vùng đề cập đến khả năng hệ thống tiếp tục hoạt động ngay cả khi xảy ra sự cố phân tách mạng, tức là khi việc giao tiếp giữa các node bị gián đoạn. Nó giống như một nhóm chat: ngay cả khi một người mất kết nối, những người còn lại vẫn có thể tiếp tục trò chuyện.

Theo định lý CAP, một hệ thống phân tán chỉ có thể đảm bảo hai trong ba thuộc tính này tại cùng một thời điểm. Nếu bạn ưu tiên tính nhất quán và khả năng chịu phân vùng, bạn có thể phải đánh đổi tính sẵn sàng, và ngược lại.

Ví dụ, một hệ thống ngân hàng cần ưu tiên tính nhất quán và khả năng chịu phân vùng để đảm bảo độ chính xác tài chính, ngay cả khi điều đó đồng nghĩa với việc một số giao dịch có thể xử lý chậm hơn, tạm thời ảnh hưởng đến tính sẵn sàng.

Vì vậy, mọi quyết định thiết kế đều đi kèm với sự đánh đổi. Chẳng hạn, một hệ thống được tối ưu cho các thao tác đọc có thể hoạt động kém hiệu quả với các thao tác ghi, hoặc để đạt hiệu năng cao hơn, chúng ta có thể phải chấp nhận tăng độ phức tạp của hệ thống.

Thiết kế hệ thống không phải là việc tìm ra một giải pháp hoàn hảo, mà là tìm ra giải pháp phù hợp nhất cho từng use case cụ thể, và điều đó đòi hỏi những quyết định có cân nhắc về việc chúng ta có thể chấp nhận đánh đổi ở đâu.

Một trong những thước đo quan trọng của hệ thống là tính sẵn sàng (availability). Đây là chỉ số đo lường hiệu suất vận hành và độ tin cậy của hệ thống. Khi nói đến availability, chúng ta đang đặt câu hỏi: Hệ thống có hoạt động khi người dùng cần hay không?

Availability thường được đo bằng tỷ lệ phần trăm, với mục tiêu nổi tiếng là “5 số 9” (99.999%). Ví dụ, nếu bạn vận hành một dịch vụ quan trọng với 99.9% availability, điều đó đồng nghĩa với khoảng 8,76 giờ downtime mỗi năm. Nhưng nếu tăng thêm hai số 9 nữa, downtime chỉ còn khoảng 5 phút mỗi năm – một sự khác biệt cực kỳ lớn, đặc biệt với các dịch vụ mà mỗi giây đều có giá trị.

Chúng ta thường đo availability thông qua uptime và downtime, và đây cũng là lúc SLO (Service Level Objective) và SLA (Service Level Agreement) phát huy vai trò.

SLO giống như việc đặt ra mục tiêu cho hiệu suất và độ sẵn sàng của hệ thống. Ví dụ, chúng ta có thể đặt một SLO rằng dịch vụ web phải phản hồi request trong vòng 300 mili-giây với 99.9% số lần.

SLA, ngược lại, là các cam kết mang tính hợp đồng chính thức với người dùng hoặc khách hàng. Nó xác định mức dịch vụ tối thiểu mà chúng ta cam kết cung cấp. Nếu SLA đảm bảo 99.99% availability mà hệ thống không đạt được, chúng ta có thể phải hoàn tiền hoặc bồi thường cho khách hàng.

Xây dựng khả năng chống chịu (resilience) cho hệ thống có nghĩa là luôn sẵn sàng cho những điều bất ngờ. Điều này có thể bao gồm việc triển khai các hệ thống dự phòng, đảm bảo luôn có một thành phần thay thế khi xảy ra lỗi, hoặc thiết kế hệ thống theo hướng suy giảm có kiểm soát (graceful degradation), để dù một số tính năng không khả dụng thì chức năng cốt lõi vẫn hoạt động.

Để đánh giá khía cạnh này, chúng ta sử dụng các khái niệm như reliability (độ tin cậy), fault tolerance (khả năng chịu lỗi) và redundancy (dự phòng).

Reliability đảm bảo hệ thống hoạt động đúng đắn và ổn định theo thời gian. Fault tolerance là việc chuẩn bị cho các tình huống lỗi – hệ thống sẽ phản ứng thế nào khi có sự cố hoặc tấn công xảy ra. Redundancy là việc có các bản sao dự phòng, đảm bảo nếu một phần của hệ thống bị lỗi thì vẫn có phần khác sẵn sàng thay thế.

Chúng ta cũng cần đo tốc độ của hệ thống, và để làm điều đó, ta sử dụng hai chỉ số chính là throughput và latency.

Throughput đo lượng dữ liệu mà hệ thống có thể xử lý trong một khoảng thời gian nhất định. Chúng ta có throughput của server, thường được đo bằng số request mỗi giây (RPS). Chỉ số này cho biết server có thể xử lý bao nhiêu request từ client trong một đơn vị thời gian. Giá trị RPS cao thường đồng nghĩa với hiệu năng tốt và khả năng phục vụ nhiều người dùng đồng thời.

Chúng ta cũng có throughput của database, được đo bằng số query mỗi giây (QPS). Chỉ số này thể hiện số lượng truy vấn mà database có thể xử lý trong một giây. Tương tự như RPS, QPS cao thường phản ánh hiệu năng tốt hơn.

Ngoài ra còn có data throughput, được đo bằng số byte mỗi giây, phản ánh lượng dữ liệu được truyền qua mạng hoặc được xử lý bởi hệ thống trong một khoảng thời gian nhất định.

Ngược lại, latency đo thời gian cần thiết để xử lý một request đơn lẻ, tức là thời gian từ lúc gửi request đến khi nhận được phản hồi. Việc tối ưu một trong hai chỉ số này thường dẫn đến sự đánh đổi với chỉ số còn lại. Ví dụ, việc gom nhóm (batching) các thao tác có thể tăng throughput nhưng lại làm tăng latency.

Thiết kế hệ thống kém có thể dẫn đến rất nhiều vấn đề về sau, từ các nút thắt hiệu năng cho đến các lỗ hổng bảo mật. Không giống như code – vốn có thể refactor tương đối dễ dàng – việc thiết kế lại toàn bộ hệ thống là một nhiệm vụ cực kỳ khó khăn và tốn kém.

Chính vì vậy, việc đầu tư thời gian và nguồn lực để thiết kế đúng ngay từ đầu là vô cùng quan trọng, nhằm xây dựng một nền móng vững chắc có thể chịu được sức nặng của các tính năng trong tương lai và sự tăng trưởng của người dùng.

Networking (TCP, UDP, DNS, IP Addresses & IP Headers)

Bây giờ, hãy cùng nói về những kiến thức cơ bản của mạng máy tính (Networking Basics). Khi đề cập đến networking cơ bản, về bản chất chúng ta đang nói về cách các máy tính giao tiếp với nhau.

Trọng tâm của quá trình giao tiếp này là địa chỉ IP, một định danh duy nhất cho mỗi thiết bị trong mạng. Địa chỉ IPv4 có độ dài 32 bit, cho phép tạo ra khoảng 4 tỷ địa chỉ duy nhất. Tuy nhiên, với sự gia tăng nhanh chóng của số lượng thiết bị, chúng ta đang dần chuyển sang IPv6, sử dụng địa chỉ 128 bit, giúp mở rộng đáng kể số lượng địa chỉ IP khả dụng.

Khi hai máy tính giao tiếp qua mạng, chúng gửi và nhận các gói dữ liệu (packet). Mỗi packet chứa một IP header, trong đó bao gồm các thông tin quan trọng như địa chỉ IP của bên gửi và bên nhận, nhằm đảm bảo dữ liệu được chuyển đến đúng đích.

Quá trình này được điều phối bởi Internet Protocol (IP) – một tập hợp các quy tắc định nghĩa cách dữ liệu được gửi và nhận qua mạng. Bên cạnh tầng IP, chúng ta còn có tầng ứng dụng (Application Layer), nơi lưu trữ dữ liệu đặc thù của từng giao thức ứng dụng.

Dữ liệu trong các packet được định dạng theo giao thức ứng dụng cụ thể, chẳng hạn như HTTP cho việc duyệt web, để thiết bị nhận có thể diễn giải dữ liệu một cách chính xác.

Khi đã hiểu các khái niệm cơ bản về địa chỉ IP và packet dữ liệu, chúng ta có thể đi sâu vào tầng vận chuyển (Transport Layer), nơi TCP và UDP đóng vai trò chính.

TCP hoạt động ở tầng vận chuyển và đảm bảo giao tiếp đáng tin cậy. Nó giống như một người giao hàng cẩn thận, không chỉ đảm bảo gói hàng đến nơi mà còn kiểm tra xem có bị thiếu hay hỏng gì không. Mỗi packet TCP đều chứa TCP header, mang các thông tin quan trọng như số hiệu cổng (port) và các cờ điều khiển cần thiết để quản lý kết nối và luồng dữ liệu.

TCP nổi tiếng với độ tin cậy cao, đảm bảo dữ liệu được truyền đi đầy đủ và chính xác. Nó thực hiện điều này thông qua các cơ chế như sequence number để theo dõi thứ tự packet, và three-way handshake – quá trình thiết lập kết nối ổn định giữa hai thiết bị.

Ngược lại, UDP nhanh hơn nhưng kém tin cậy hơn TCP. UDP không thiết lập kết nối trước khi gửi dữ liệu và cũng không đảm bảo packet sẽ được gửi đến hay theo đúng thứ tự. Tuy nhiên, chính đặc điểm này khiến UDP phù hợp với các hình thức giao tiếp nhạy cảm về thời gian như gọi video hoặc livestream, nơi tốc độ quan trọng hơn và việc mất một phần dữ liệu có thể chấp nhận được.

Để kết nối tất cả các khái niệm trên, hãy nói về DNS (Domain Name System). DNS hoạt động giống như “danh bạ” của Internet, chuyển đổi các tên miền thân thiện với con người thành địa chỉ IP.

Khi bạn nhập một URL vào trình duyệt, trình duyệt sẽ gửi một truy vấn DNS để tìm địa chỉ IP tương ứng, từ đó thiết lập kết nối đến server và tải nội dung trang web.

Hoạt động của DNS được giám sát bởi ICANN, tổ chức điều phối không gian địa chỉ IP toàn cầu và hệ thống tên miền. Các nhà đăng ký tên miền như Namecheap hay GoDaddy được ICANN ủy quyền để bán tên miền cho công chúng.

DNS sử dụng nhiều loại record khác nhau, chẳng hạn như A record dùng để ánh xạ tên miền sang địa chỉ IPv4, đảm bảo request của bạn được gửi đến đúng server, hoặc AAAA record dùng để ánh xạ tên miền sang địa chỉ IPv6.

Cuối cùng, hãy nói về hạ tầng mạng hỗ trợ toàn bộ quá trình giao tiếp này. Các thiết bị trong mạng có thể sử dụng IP public hoặc IP private. IP public là duy nhất trên toàn Internet, trong khi IP private chỉ duy nhất trong phạm vi một mạng nội bộ.

Địa chỉ IP có thể được gán tĩnh (static) hoặc động (dynamic). IP động thường được sử dụng cho các kết nối Internet gia đình. Các thiết bị trong cùng một mạng cục bộ (LAN) có thể giao tiếp trực tiếp với nhau.

Để bảo vệ các mạng này, chúng ta sử dụng firewall, có nhiệm vụ giám sát và kiểm soát lưu lượng mạng vào và ra. Bên trong mỗi thiết bị, các tiến trình hoặc dịch vụ cụ thể được xác định bằng port. Khi kết hợp một port với một địa chỉ IP, chúng ta sẽ có một định danh duy nhất cho một dịch vụ mạng.

Một số port được dành riêng cho các giao thức phổ biến, chẳng hạn như port 80 cho HTTP hoặc port 22 cho SSH.

Application Layer Protocols (HTTP, WebSockets, WebRTC, MQTT, etc)

Bây giờ, hãy cùng điểm qua các giao thức tầng ứng dụng (Application Layer Protocols) quan trọng nhất. Giao thức phổ biến nhất trong số này là HTTP, viết tắt của HyperText Transfer Protocol, được xây dựng trên nền TCP/IP.

HTTP là một giao thức theo mô hình request–response, nhưng bạn có thể hình dung nó như một cuộc hội thoại không có trí nhớ: mỗi lần tương tác đều độc lập và không ghi nhớ những lần trước đó. Điều này có nghĩa là server không cần lưu trạng thái hay ngữ cảnh giữa các request; thay vào đó, mỗi request đều phải chứa đầy đủ thông tin cần thiết.

Trong HTTP, header chứa các thông tin như URL và method, trong khi body mang nội dung chính của request hoặc response. Mỗi response cũng bao gồm một status code, dùng để phản hồi kết quả xử lý request của client trên server.

Các status code được chia thành nhiều nhóm:

- 200 series là các mã thành công, cho biết request đã được nhận và xử lý thành công.
- 300 series là các mã chuyển hướng, cho biết client cần thực hiện thêm hành động để hoàn tất request.
- 400 series là các mã lỗi phía client, xảy ra khi request có cú pháp sai hoặc không thể được xử lý.
- 500 series là các mã lỗi phía server, cho biết đã có sự cố xảy ra trên server.

Mỗi request HTTP cũng có một method. Những method phổ biến nhất là GET, POST, PUT, PATCH và DELETE.

GET dùng để lấy dữ liệu, POST thường dùng để tạo dữ liệu mới trên server, PUT và PATCH dùng để cập nhật dữ liệu, còn DELETE dùng để xóa dữ liệu khỏi cơ sở dữ liệu.

HTTP là kết nối một chiều, nhưng để phục vụ các cập nhật thời gian thực, chúng ta sử dụng WebSocket. WebSocket cung cấp kênh giao tiếp hai chiều trên một kết nối lâu dài duy nhất, cho phép server chủ động đẩy dữ liệu real-time xuống client.

Điều này đặc biệt quan trọng với các ứng dụng cần cập nhật liên tục mà không phải lặp đi lặp lại các chu kỳ request–response của HTTP. WebSocket thường được dùng trong các ứng dụng chat, cập nhật tỷ số thể thao trực tiếp hoặc dữ liệu thị trường chứng khoán, nơi hoạt động diễn ra liên tục và cuộc hội thoại không bao giờ dừng lại.

Đối với các giao thức liên quan đến email, SMTP là tiêu chuẩn để truyền email qua Internet. Đây là giao thức dùng để gửi email giữa các server. Hầu hết các email client đều sử dụng SMTP để gửi email, và dùng IMAP hoặc POP3 để nhận email.

IMAP cho phép client truy cập và thao tác trực tiếp với email trên server, rất phù hợp cho những người dùng cần truy cập email từ nhiều thiết bị khác nhau. POP3 dùng để tải email từ server về máy cục bộ, thường được sử dụng khi email chỉ được quản lý trên một thiết bị duy nhất.

Chuyển sang các giao thức truyền và quản lý file, giao thức truyền thông để truyền file qua Internet là FTP. FTP thường được dùng trong việc bảo trì website và truyền các tập dữ liệu lớn, cho phép chuyển file giữa client và server, chẳng hạn như upload file lên server hoặc sao lưu dữ liệu.

Bên cạnh đó, chúng ta có SSH (Secure Shell), một giao thức dùng để vận hành các dịch vụ mạng một cách an toàn trên môi trường mạng không bảo mật. SSH thường được sử dụng để đăng nhập vào máy chủ từ xa, thực thi lệnh hoặc truyền file một cách bảo mật.

Ngoài ra còn có các giao thức phục vụ giao tiếp thời gian thực như WebRTC, cho phép các ứng dụng trình duyệt giao tiếp trực tiếp với nhau để gọi thoại, gọi video và chia sẻ file mà không cần plugin bên trong hay bên ngoài. Đây là nền tảng quan trọng cho các ứng dụng họp video và livestream.

Một giao thức khác là MQTT, một giao thức nhắn tin nhẹ, rất phù hợp cho các thiết bị có tài nguyên hạn chế và các kịch bản yêu cầu băng thông thấp, chẳng hạn như các thiết bị IoT.

AMQP là một giao thức dành cho middleware hướng thông điệp, cung cấp độ tin cậy và bảo mật cao cho giao tiếp ở cấp độ doanh nghiệp, ví dụ như trong các công cụ như RabbitMQ.

Cuối cùng, hãy nói về RPC (Remote Procedure Call). Đây là một giao thức cho phép một chương trình trên máy tính này thực thi một đoạn code trên server hoặc một máy

tính khác. RPC cho phép gọi một hàm như thể nó là hàm cục bộ, trong khi thực tế hàm đó được thực thi trên một máy từ xa.

RPC che giấu các chi tiết phức tạp của giao tiếp mạng, cho phép lập trình viên tương tác với các hàm từ xa một cách liền mạch, giống như đang làm việc với các hàm trong cùng một ứng dụng. Nhiều giao thức tầng ứng dụng sử dụng cơ chế RPC để thực hiện các thao tác của mình.

Ví dụ, trong các web service, một HTTP request có thể dẫn đến các RPC call ở backend để xử lý dữ liệu hoặc thực hiện hành động thay mặt client. Tương tự, các SMTP server cũng có thể sử dụng RPC nội bộ để xử lý email hoặc tương tác với cơ sở dữ liệu.

API Design

Tất nhiên, vẫn còn rất nhiều giao thức tầng ứng dụng khác, nhưng những giao thức được đề cập ở đây là những giao thức phổ biến và thiết yếu nhất cho phát triển web. Trong phần này, chúng ta sẽ cùng tìm hiểu về thiết kế API, bắt đầu từ những khái niệm cơ bản và tiến dần đến các best practice giúp tạo ra những API chất lượng cao.

Hãy xét một API cho một nền tảng thương mại điện tử như Shopify. Nếu bạn chưa quen thuộc, Shopify là một nền tảng thương mại điện tử nổi tiếng, cho phép các doanh nghiệp thiết lập cửa hàng trực tuyến. Trong thiết kế API, chúng ta quan tâm đến việc xác định đầu vào, chẳng hạn như thông tin sản phẩm mới do người bán cung cấp, và đầu ra, ví dụ như dữ liệu được trả về khi có người truy vấn thông tin sản phẩm thông qua API.

Trọng tâm của API design là cách các thao tác CRUD được cung cấp cho giao diện người dùng. CRUD là viết tắt của Create, Read, Update, Delete, những thao tác cơ bản của bất kỳ ứng dụng hướng dữ liệu nào.

Ví dụ, để thêm một sản phẩm mới, chúng ta gửi một POST request đến endpoint `/api/products`, trong đó thông tin sản phẩm được truyền trong request body. Để lấy danh sách sản phẩm, ta gửi GET request đến `/api/products`. Để cập nhật sản phẩm, ta sử dụng PUT hoặc PATCH request đến `/product/{id}` – nơi `{id}` là ID của sản phẩm cần cập nhật. Việc xóa sản phẩm cũng tương tự, sử dụng endpoint `/product/{id}` cho thao tác DELETE.

Ngoài ra, chúng ta cũng có thể có một GET request đến `/product/{id}` để lấy thông tin chi tiết của một sản phẩm đơn lẻ.

Một phần quan trọng khác trong thiết kế API là quyết định giao thức giao tiếp sẽ sử dụng, chẳng hạn như HTTP, WebSocket hoặc các giao thức khác, cũng như cơ chế truyền dữ liệu, có thể là JSON, XML hoặc Protocol Buffers.

Điều này thường áp dụng cho RESTful API, nhưng bên cạnh đó chúng ta còn có các mô hình khác như GraphQL và gRPC. API có thể tồn tại dưới nhiều paradigm khác nhau, mỗi paradigm có bộ tiêu chuẩn và giao thức riêng.

Mô hình phổ biến nhất là REST, viết tắt của Representational State Transfer. REST là stateless, nghĩa là mỗi request từ client đến server phải chứa đầy đủ thông tin cần thiết để server hiểu và xử lý request đó. REST sử dụng các HTTP method tiêu chuẩn như GET, POST, PUT và DELETE, và rất dễ được tiêu thụ bởi nhiều loại client khác nhau như trình duyệt web hoặc ứng dụng mobile.

Nhược điểm của RESTful API là dễ dẫn đến over-fetching hoặc under-fetching dữ liệu, do có thể cần nhiều endpoint khác nhau để lấy đúng dữ liệu mong muốn. RESTful API thường sử dụng JSON làm định dạng trao đổi dữ liệu.

Ngược lại, GraphQL API cho phép client yêu cầu chính xác những dữ liệu mà họ cần, giúp tránh tình trạng over-fetching và under-fetching. GraphQL có các query được định kiểu chặt chẽ (strongly typed), nhưng các query phức tạp có thể ảnh hưởng đến hiệu năng của server. Tất cả các request GraphQL đều được gửi dưới dạng POST request, và GraphQL thường trả về HTTP status code 200 ngay cả khi có lỗi, với chi tiết lỗi được chứa trong response body.

gRPC là viết tắt của Google Remote Procedure Call, được xây dựng trên HTTP/2, cung cấp các tính năng nâng cao như multiplexing và server push. gRPC sử dụng Protocol Buffers để serialize dữ liệu có cấu trúc, nhờ đó rất hiệu quả về băng thông và tài nguyên, đặc biệt phù hợp cho kiến trúc microservices.

Nhược điểm của gRPC là dữ liệu kém thân thiện với con người hơn so với JSON, và nó yêu cầu môi trường hỗ trợ HTTP/2 để có thể hoạt động.

Trong bối cảnh thương mại điện tử, bạn thường có các mối quan hệ dữ liệu như user – orders hoặc orders – products, và bạn cần thiết kế các endpoint phản ánh những mối quan hệ này. Ví dụ, để lấy danh sách đơn hàng của một user cụ thể, bạn có thể gửi request đến `/users/{userId}/orders`.

Các query phổ biến khác bao gồm limit và offset để phân trang, hoặc start date và end date để lọc sản phẩm trong một khoảng thời gian nhất định. Điều này cho phép client chỉ lấy những tập dữ liệu cần thiết mà không gây quá tải cho hệ thống.

Một GET request được thiết kế tốt cần phải idempotent, nghĩa là việc gọi nó nhiều lần không làm thay đổi kết quả và luôn trả về cùng một dữ liệu. GET request không bao giờ được thay đổi dữ liệu; chúng chỉ dùng để truy xuất dữ liệu. Nếu cần tạo hoặc cập nhật dữ liệu, bạn phải sử dụng POST hoặc PUT request.

Khi chỉnh sửa hoặc mở rộng các endpoint, việc duy trì backward compatibility là cực kỳ quan trọng. Điều này có nghĩa là các thay đổi mới không được làm hỏng các client hiện tại. Một cách làm phổ biến là giới thiệu version mới của API, chẳng hạn như /v2/products, trong khi version 1 vẫn tiếp tục phục vụ các client cũ, và version 2 phục vụ các client mới.

Đối với RESTful API, cách làm này rất phổ biến. Trong trường hợp của GraphQL, việc thêm các field mới mà không xóa field cũ cho phép API phát triển mà không làm gián đoạn các client hiện có.

Một best practice khác là thiết lập rate limiting. Điều này giúp ngăn chặn các cuộc tấn công DDoS và kiểm soát số lượng request mà một người dùng có thể gửi trong một khoảng thời gian nhất định, tránh việc một client đơn lẻ gửi quá nhiều request làm quá tải API.

Cuối cùng, một thực hành quan trọng khác là cấu hình CORS (Cross-Origin Resource Sharing). Với CORS, bạn có thể kiểm soát những domain nào được phép truy cập vào API của mình, từ đó ngăn chặn các tương tác cross-site không mong muốn.

Caching and CDNs

Độ trễ và vấn đề khoảng cách địa lý:

Hãy tưởng tượng một công ty đang lưu trữ website của mình trên một máy chủ đặt tại trung tâm dữ liệu Google Cloud ở Phần Lan. Website này có thể chỉ mất khoảng 100 mili-giây để tải đối với người dùng ở châu Âu, nhưng lại mất từ 3 đến 5 giây đối với người dùng ở Mexico.

May mắn là có những chiến lược giúp giảm độ trễ của các yêu cầu từ người dùng ở xa. Những chiến lược này được gọi là caching (bộ nhớ đệm) và CDN (Content Delivery Network – mạng phân phối nội dung). Đây là hai khái niệm rất quan trọng trong phát triển web hiện đại và thiết kế hệ thống.

Khái niệm caching:

Caching là một kỹ thuật dùng để cải thiện hiệu năng và hiệu quả của hệ thống. Nó hoạt động bằng cách lưu trữ một bản sao của một số dữ liệu trong bộ nhớ tạm, để các yêu cầu sau này đối với dữ liệu đó có thể được phục vụ nhanh hơn.

Browser caching (cache phía trình duyệt):

Một trong những nơi phổ biến nhất để lưu cache là trình duyệt. Ở đây, các tài nguyên của website được lưu trên máy tính cục bộ của người dùng. Khi người dùng truy cập lại website, trình duyệt có thể tải dữ liệu trực tiếp từ cache thay vì phải tải lại toàn bộ từ server.

Người dùng có thể tắt cache bằng cách điều chỉnh cài đặt trình duyệt. Nhà phát triển cũng có thể tắt cache trong công cụ dành cho lập trình viên, ví dụ như tùy chọn Disable cache trong tab Network của Chrome DevTools.

Cache của trình duyệt được lưu trong một thư mục trên ổ cứng của người dùng và do trình duyệt quản lý. Thông thường, browser cache lưu các file HTML, CSS và JavaScript bundle. Chúng ta sử dụng header Cache-Control để chỉ định thời gian nội dung được cache, ví dụ 7.200 giây (tương đương 2 giờ).

Cache hit, cache miss và cache ratio:

Khi dữ liệu được yêu cầu có sẵn trong cache, ta gọi đó là cache hit. Ngược lại, nếu dữ liệu không có trong cache và hệ thống phải lấy từ nguồn gốc, ta gọi đó là cache miss. Cache ratio là tỷ lệ phần trăm các request được phục vụ từ cache so với tổng số request. Tỷ lệ này càng cao thì hệ thống cache càng hiệu quả. Ta có thể kiểm tra cache hit hay miss thông qua header X-Cache trong response.

Server-side caching (cache phía server):

Server caching là việc lưu trữ dữ liệu được truy cập thường xuyên ngay tại phía server, nhằm giảm các thao tác tốn kém như truy vấn database. Cache phía server có thể được lưu trực tiếp trên server hoặc trên một cache server riêng, có thể là trong bộ nhớ (như Redis) hoặc trên ổ đĩa.

Thông thường, server sẽ kiểm tra cache trước khi truy vấn database. Nếu dữ liệu có trong cache, nó sẽ được trả về ngay. Nếu không, server sẽ truy vấn database, trả kết quả cho người dùng và đồng thời lưu kết quả đó vào cache cho những lần truy cập sau.

Các chiến lược ghi cache:

Trong mô hình write-around cache, dữ liệu được ghi trực tiếp vào bộ nhớ lưu trữ lâu dài (database) mà không đi qua cache. Cách này thường dùng khi hiệu năng ghi không quá quan trọng.

Với write-through cache, dữ liệu được ghi đồng thời vào cache và bộ nhớ lâu dài, giúp đảm bảo tính nhất quán nhưng có thể chậm hơn.

Còn write-back cache thì dữ liệu được ghi vào cache trước và ghi xuống bộ nhớ lâu dài sau, giúp cải thiện hiệu năng ghi nhưng có rủi ro mất dữ liệu nếu server bị crash.

Eviction policies (chính sách loại bỏ cache):

Khi cache đầy, hệ thống cần loại bỏ bớt dữ liệu để nhường chỗ cho dữ liệu mới. Việc này được thực hiện dựa trên các chính sách loại bỏ như:

- Loại bỏ dữ liệu ít được truy cập gần đây nhất (LRU)
- Loại bỏ dữ liệu được thêm vào sớm nhất (FIFO)
- Loại bỏ dữ liệu ít được sử dụng nhất (LFU)

Database caching:

Database caching là việc cache kết quả của các truy vấn database để cải thiện hiệu năng của các ứng dụng phụ thuộc nhiều vào dữ liệu. Việc này có thể được thực hiện ngay trong database hoặc thông qua một lớp cache bên ngoài như Redis hay Memcached. Khi có truy vấn, hệ thống sẽ kiểm tra cache trước. Nếu có kết quả, nó sẽ trả về ngay. Nếu không, truy vấn sẽ được thực thi trên database và kết quả sẽ được lưu vào cache để dùng cho các lần sau. Cách này đặc biệt hiệu quả với các ứng dụng có lượng đọc lớn.

CDN (Content Delivery Network):

Một hình thức caching khác là CDN – mạng lưới các server được phân bố trên nhiều khu vực địa lý. CDN thường được dùng để phục vụ nội dung tĩnh như JavaScript, HTML, CSS, hình ảnh và video.

Khi người dùng yêu cầu một tài nguyên, request sẽ được chuyển đến CDN server gần nhất. Nếu server đó đã có dữ liệu, nó sẽ trả về ngay. Nếu chưa, CDN sẽ lấy dữ liệu từ origin server, lưu lại và gửi cho người dùng.

Các loại CDN:

Với pull-based CDN, CDN tự động lấy nội dung từ origin server khi có request đầu tiên. Cách này phù hợp với website có nhiều nội dung tĩnh được cập nhật thường xuyên và yêu cầu ít quản lý thủ công.

Trong push-based CDN, nội dung được upload chủ động lên CDN. Cách này phù hợp với các file lớn, ít thay đổi nhưng cần được phân phối nhanh khi cập nhật, tuy nhiên đòi hỏi quản lý nhiều hơn.

Khi nào vẫn cần origin server:

Trong một số trường hợp, request vẫn phải đi đến origin server, chẳng hạn như khi xử lý nội dung động thay đổi liên tục, các tác vụ thời gian thực hoặc các logic server phức tạp mà CDN không thể xử lý.

Lợi ích của CDN và caching:

CDN giúp giảm độ trễ bằng cách phục vụ nội dung từ vị trí gần người dùng hơn. Nó cũng tăng khả năng mở rộng, tính sẵn sàng cao và khả năng chịu tải, đồng thời cải thiện bảo mật nhờ các tính năng như chống DDoS và mã hóa traffic.

Caching nói chung giúp truy xuất dữ liệu nhanh hơn, giảm tải cho server chính và mang lại thời gian tải trang nhanh hơn, từ đó cải thiện trải nghiệm người dùng tổng thể.

Proxy Servers (Forward/Reverse Proxies)

Bây giờ, chúng ta hãy nói về proxy server, một thành phần hoạt động như trung gian giữa client (bên yêu cầu tài nguyên) và server (bên cung cấp tài nguyên). Proxy server có thể phục vụ nhiều mục đích khác nhau như cache tài nguyên để truy cập nhanh hơn, ẩn danh các request, và cân bằng tải giữa nhiều server. Về bản chất, proxy nhận request từ client, chuyển tiếp chúng đến các server phù hợp, rồi trả lại response từ server về cho client.

Có nhiều loại proxy server khác nhau, mỗi loại phục vụ một mục đích riêng. Dưới đây là một số loại chính.

Loại đầu tiên là forward proxy. Forward proxy nằm ở phía trước client và được dùng để gửi request từ client đến các server khác trên Internet. Nó thường được sử dụng trong các mạng nội bộ để kiểm soát việc truy cập Internet.

Tiếp theo là reverse proxy. Reverse proxy nằm ở phía trước một hoặc nhiều web server, chặn các request từ Internet trước khi chúng đến server. Nó được dùng cho cân bằng tải, tăng tốc web, và hoạt động như một lớp bảo mật.

Một loại khác là open proxy, cho phép bất kỳ người dùng nào kết nối và sử dụng proxy server. Loại này thường được dùng để ẩn danh việc duyệt web và vượt qua các giới hạn nội dung.

Chúng ta cũng có transparent proxy. Loại proxy này chuyển tiếp request và tài nguyên mà không chỉnh sửa, nhưng nó hiển thị rõ với client. Transparent proxy thường được dùng cho cache và lọc nội dung.

Tiếp theo là anonymous proxy. Proxy này có thể bị nhận diện là proxy server, nhưng không tiết lộ địa chỉ IP gốc của client. Nó được sử dụng cho mục đích duyệt web ẩn danh.

Một loại khác là distorting proxy, proxy này cung cấp địa chỉ IP gốc sai cho server đích. Nó tương tự anonymous proxy nhưng cố tình cung cấp thông tin IP không chính xác.

Loại phổ biến tiếp theo là high anonymity proxy (hay elite proxy). Loại proxy này khiến việc phát hiện proxy trở nên rất khó khăn. Chúng không gửi header như X-Forwarded-For hay các header nhận dạng khác, đảm bảo mức độ ẩn danh tối đa.

Trong thực tế, forward proxy và reverse proxy là hai loại được sử dụng phổ biến nhất.

Forward proxy hoạt động như một lớp trung gian giữa client và server. Nó nằm giữa client (có thể là một máy tính trong mạng nội bộ) và các server bên ngoài (ví dụ như các website trên Internet). Khi client gửi request, request đó sẽ được gửi đến forward proxy trước. Proxy sẽ đánh giá request và, dựa trên cấu hình và các rule của nó, quyết định cho phép, chỉnh sửa, hay chặn request.

Một trong những chức năng quan trọng nhất của forward proxy là che giấu địa chỉ IP của client. Khi proxy chuyển tiếp request đến server đích, server sẽ thấy request như thể nó đến từ proxy server chứ không phải từ client ban đầu.

Hãy xem một vài use case phổ biến của forward proxy.

Một ví dụ nổi tiếng là Instagram proxy. Đây là một dạng forward proxy đặc biệt, được dùng để quản lý nhiều tài khoản Instagram mà không bị khóa hay giới hạn. Marketer và social media manager sử dụng Instagram proxy để tạo cảm giác như họ đang ở những khu vực khác nhau hoặc là những người dùng khác nhau, từ đó có thể quản lý nhiều tài khoản, tự động hóa tác vụ, hoặc thu thập dữ liệu mà không bị đánh dấu là hoạt động đáng ngờ.

Một ví dụ khác là kiểm soát và giám sát việc sử dụng Internet. Một số tổ chức dùng forward proxy để theo dõi và kiểm soát việc truy cập Internet của nhân viên. Proxy có thể chặn các website không liên quan đến công việc và bảo vệ khỏi các mối đe dọa trên web. Chúng cũng có thể quét virus và malware trong nội dung đi vào hệ thống.

Một use case phổ biến nữa là caching nội dung được truy cập thường xuyên. Forward proxy có thể cache các website hoặc nội dung phổ biến, giúp giảm băng thông và tăng tốc truy cập cho người dùng trong mạng. Điều này đặc biệt hữu ích trong các mạng có băng thông hạn chế hoặc chi phí cao.

Forward proxy cũng có thể được dùng để ẩn danh việc truy cập web. Những người quan tâm đến quyền riêng tư có thể sử dụng forward proxy để che giấu địa chỉ IP và các thông tin nhận dạng khác khi truy cập website, khiến việc theo dõi hành vi duyệt web trở nên khó khăn hơn.

Ngược lại, reverse proxy là một loại proxy server nằm phía trước một hoặc nhiều web server, chặn request từ client trước khi chúng đến server. Trong khi forward proxy che giấu danh tính của client, thì reverse proxy che giấu danh tính của server, hoặc thậm chí là sự tồn tại của nhiều server phía sau.

Client chỉ tương tác với reverse proxy và có thể không hề biết về các server phía sau nó. Reverse proxy cũng phân phối request của client đến nhiều server khác nhau, giúp cân bằng tải và đảm bảo không server nào bị quá tải.

Reverse proxy còn có thể nén dữ liệu vào và ra, cache file, và quản lý mã hóa SSL, từ đó giúp tăng tốc thời gian tải và giảm tải cho server.

Một số use case phổ biến của reverse proxy bao gồm load balancer. Load balancer phân phối traffic mạng đến nhiều server, đảm bảo không server nào nhận quá nhiều tải. Việc phân phối traffic này giúp tránh tình trạng nghẽn cổ chai và duy trì tốc độ cũng như độ ổn định của dịch vụ.

CDN (Content Delivery Network) cũng là một dạng reverse proxy. CDN là một mạng lưới server phân phối nội dung tĩnh đã được cache đến người dùng dựa trên vị trí địa lý. CDN hoạt động như reverse proxy bằng cách lấy nội dung từ origin server, cache lại, và đưa nội dung đến gần người dùng hơn để truyền tải nhanh hơn.

Một ví dụ khác là Web Application Firewall (WAF). WAF được đặt phía trước các web application để kiểm tra traffic đầu vào, chặn các cuộc tấn công hack và lọc traffic không mong muốn. Firewall cũng bảo vệ ứng dụng khỏi các lỗ hổng web phổ biến.

Ví dụ cuối cùng là SSL offloading (hoặc SSL acceleration). Một số reverse proxy đảm nhận việc mã hóa và giải mã SSL/TLS, giúp giảm tải công việc này cho web server và tối ưu hiệu năng tổng thể.

Load Balancers

Load balancer có lẽ là use case phổ biến nhất của proxy server. Chúng phân phối traffic đầu vào đến nhiều server khác nhau để đảm bảo không server nào phải gánh quá nhiều tải. Bằng cách phân tán các request một cách hiệu quả, load balancer giúp tăng khả năng mở rộng (capacity) và độ tin cậy (reliability) của ứng dụng.

Dưới đây là một số chiến lược và thuật toán phổ biến được sử dụng trong load balancing.

Thuật toán đầu tiên là Round Robin. Đây là hình thức load balancing đơn giản nhất, trong đó mỗi server trong pool sẽ lần lượt nhận request theo thứ tự xoay vòng. Khi đến server cuối cùng, hệ thống sẽ quay lại server đầu tiên. Thuật toán này hoạt động tốt khi các server có cấu hình tương tự nhau và tải có thể phân phối đồng đều.

Tiếp theo là thuật toán Least Connections. Thuật toán này điều hướng traffic đến server có ít kết nối đang hoạt động nhất. Nó rất phù hợp với các tác vụ chạy lâu hoặc trong trường hợp tải giữa các server không phân bố đều.

Một thuật toán khác là Least Response Time. Thuật toán này chọn server có thời gian phản hồi thấp nhất và ít kết nối đang hoạt động nhất. Mục tiêu của thuật toán này là đem lại tốc độ phản hồi nhanh nhất cho các request.

Tiếp theo là IP Hashing. Thuật toán này quyết định server nào sẽ nhận request dựa trên giá trị hash của địa chỉ IP client. Cách làm này đảm bảo một client sẽ luôn kết nối đến cùng một server, rất hữu ích cho session persistence trong các ứng dụng cần duy trì trạng thái phiên làm việc.

Các biến thể của những phương pháp trên cũng thường được sử dụng, dẫn đến khái niệm weighted algorithms. Ví dụ như Weighted Round Robin hoặc Weighted Least

Connections. Trong các thuật toán này, mỗi server được gán một trọng số (weight), thường dựa trên năng lực xử lý hoặc hiệu năng. Các server mạnh hơn sẽ xử lý nhiều request hơn. Cách này hiệu quả khi các server trong pool có khả năng khác nhau, ví dụ CPU hoặc RAM khác nhau.

Ngoài ra còn có thuật toán dựa trên vị trí địa lý (Geographical algorithms). Các thuật toán này điều hướng request đến server gần vị trí địa lý của người dùng nhất hoặc dựa trên các yêu cầu vùng miền cụ thể. Điều này đặc biệt hữu ích cho các dịch vụ toàn cầu, nơi việc giảm độ trễ là ưu tiên hàng đầu.

Một thuật toán phổ biến khác là Consistent Hashing. Thuật toán này sử dụng một hàm hash để phân phối dữ liệu lên nhiều node khác nhau. Ta có thể hình dung một không gian hash dạng vòng tròn, trong đó điểm cuối nối lại điểm đầu, thường được gọi là hash ring. Cả các node (server) và dữ liệu (ví dụ như key hoặc giá trị lưu trữ) đều được hash lên vòng tròn này. Cách tiếp cận này đảm bảo client luôn kết nối đến cùng một server, và là một đặc tính quan trọng trong thiết kế load balancer.

Một tính năng thiết yếu của load balancer là health check liên tục đối với các server phía sau. Điều này đảm bảo traffic chỉ được gửi đến các server đang hoạt động và phản hồi tốt. Nếu một server bị lỗi, load balancer sẽ ngừng gửi traffic đến server đó cho đến khi nó hoạt động trở lại.

Load balancer có thể tồn tại dưới nhiều hình thức khác nhau, bao gồm phần cứng (hardware), phần mềm (software) và dịch vụ trên nền tảng cloud.

Một số hardware load balancer phổ biến bao gồm F5 BIG-IP, một thiết bị load balancer phần cứng được sử dụng rộng rãi, nổi tiếng với hiệu năng cao và bộ tính năng phong phú. Nó cung cấp các chức năng như local traffic management, global server load balancing, và bảo mật ứng dụng.

Một ví dụ khác là Citrix ADC (trước đây gọi là NetScaler), cung cấp các chức năng load balancing, content switching, và tăng tốc ứng dụng.

Về software load balancer, một cái tên phổ biến là HAProxy, một phần mềm mã nguồn mở rất được ưa chuộng, hoạt động như load balancer và proxy server cho các ứng dụng dựa trên TCP và HTTP.

Nginx cũng là một lựa chọn rất phổ biến. Dù thường được dùng như web server, Nginx còn hoạt động như một load balancer và reverse proxy cho HTTP và nhiều giao thức mạng khác.

Trong lĩnh vực cloud-based load balancer, có thể kể đến AWS Elastic Load Balancing, Microsoft Azure Load Balancer, và Google Cloud Load Balancer.

Ngoài ra còn có các virtual load balancer, ví dụ như VMware NSX Advanced Load Balancer (Avi Networks). Giải pháp này cung cấp một Application Delivery Controller (ADC) dạng software-defined, có thể triển khai on-premise hoặc trên cloud.

Bây giờ, hãy xem điều gì sẽ xảy ra khi load balancer bị sập. Khi load balancer gặp sự cố, nó có thể ảnh hưởng đến toàn bộ khả năng hoạt động và hiệu năng của ứng dụng hoặc dịch vụ mà nó quản lý. Về bản chất, load balancer có thể trở thành single point of failure. Khi nó ngừng hoạt động, tất cả các server phía sau cũng trở nên không thể truy cập được đối với client.

Để tránh hoặc giảm thiểu tác động của việc load balancer bị lỗi, có một số chiến lược có thể áp dụng.

Chiến lược đầu tiên là triển khai load balancer dự phòng (redundant load balancing) bằng cách sử dụng nhiều load balancer, thường là theo cặp. Nếu một load balancer gặp sự cố, load balancer còn lại sẽ tiếp quản, đây là cơ chế được gọi là failover.

Chiến lược tiếp theo là giám sát và health check liên tục chính load balancer. Điều này giúp phát hiện sớm các vấn đề và xử lý trước khi chúng gây ra gián đoạn nghiêm trọng.

Chúng ta cũng có thể triển khai auto-scaling và self-healing systems. Nhiều hạ tầng hiện đại được thiết kế để tự động phát hiện lỗi của load balancer và thay thế nó bằng một instance mới mà không cần can thiệp thủ công.

Trong một số cấu hình, DNS failover có thể chuyển hướng traffic ra khỏi một địa chỉ IP không còn nhận kết nối (ví dụ như load balancer bị lỗi) sang một IP dự phòng đã được cấu hình sẵn, đây chính là load balancer mới.

Databases (Sharding, Replication, ACID, Vertical & Horizontal Scaling)

Các buổi phỏng vấn system design sẽ không thể trọn vẹn nếu thiếu một phần tìm hiểu sâu về database. Trong vài phút tiếp theo, tôi sẽ dẫn bạn đi qua những kiến thức nền tảng về database mà bạn cần nắm vững để vượt qua buổi phỏng vấn. Chúng ta sẽ khám phá vai trò của database trong system design, các kỹ thuật sharding và replication, cùng với những thuộc tính ACID quan trọng.

Chúng ta cũng sẽ thảo luận về các loại database khác nhau, các lựa chọn scale theo chiều dọc và chiều ngang, và những kỹ thuật tối ưu hiệu năng database.

Có nhiều loại database khác nhau, mỗi loại được thiết kế để giải quyết những bài toán và thách thức riêng. Hãy cùng khám phá từng loại.

Loại đầu tiên là relational database (cơ sở dữ liệu quan hệ). Bạn có thể hình dung relational database giống như một tủ hồ sơ được sắp xếp rất ngăn nắp, nơi mọi dữ liệu được phân loại rõ ràng vào các ngăn kéo và thư mục khác nhau. Một số ví dụ phổ biến của SQL database bao gồm PostgreSQL, MySQL và SQLite.

Tất cả các SQL database đều sử dụng bảng (tables) để lưu trữ dữ liệu và dùng SQL làm ngôn ngữ truy vấn. Chúng rất phù hợp cho giao dịch (transactions), truy vấn phức tạp, và đảm bảo tính toàn vẹn dữ liệu.

Relational database cũng tuân thủ ACID, nghĩa là chúng đảm bảo các thuộc tính ACID.

- A – Atomicity (tính nguyên tử): giao dịch phải thực hiện toàn bộ hoặc không thực hiện gì cả.
- C – Consistency (tính nhất quán): sau mỗi giao dịch, database phải ở trạng thái nhất quán.
- I – Isolation (tính cô lập): các giao dịch phải độc lập với nhau.
- D – Durability (tính bền vững): khi giao dịch đã commit, dữ liệu sẽ được lưu trữ vĩnh viễn.

Chúng ta cũng có NoSQL database, thường hy sinh tính nhất quán (Consistency) trong ACID để đạt được khả năng mở rộng tốt hơn. Bạn có thể hình dung NoSQL database giống như một bảng brainstorm với các sticky note, nơi bạn có thể thêm hoặc xóa dữ liệu với bất kỳ hình dạng nào — rất linh hoạt.

Một số ví dụ phổ biến của NoSQL database là MongoDB, Cassandra và Redis. Có nhiều loại NoSQL database khác nhau như:

- Key-value store (ví dụ: Redis)
- Document-based database (ví dụ: MongoDB)
- Graph-based database (ví dụ: Neo4j)

NoSQL database là schema-less, nghĩa là chúng không có foreign key để liên kết dữ liệu như relational database. Chúng rất phù hợp cho dữ liệu phi cấu trúc, lý tưởng cho khả năng mở rộng, phát triển nhanh, và các truy vấn đơn giản.

Ngoài ra còn có in-memory database. Bạn có thể tưởng tượng nó giống như một bảng trắng để làm các phép tính nhanh hoặc vẽ phác thảo tạm thời. Vì mọi dữ liệu đều nằm trong bộ nhớ nên tốc độ xử lý rất nhanh. Một số ví dụ là Redis và Memcached.

In-memory database có khả năng truy xuất dữ liệu cực nhanh, và thường được dùng chủ yếu cho caching và lưu trữ session.

Bây giờ, hãy cùng xem cách scale database.

Lựa chọn đầu tiên là vertical scaling (scale up). Với vertical scaling, bạn cải thiện hiệu năng database bằng cách nâng cấp tài nguyên của server hiện tại, chẳng hạn như tăng CPU, thêm RAM, sử dụng ổ đĩa nhanh hơn hoặc nâng cấp network.

Tuy nhiên, cách này có giới hạn tối đa, vì bạn chỉ có thể nâng cấp tài nguyên của một máy đến một mức nhất định. Do đó, vertical scaling rất hạn chế về lâu dài.

Lựa chọn tiếp theo là horizontal scaling (scale out), tức là thêm nhiều máy vào hệ thống thay vì nâng cấp một máy duy nhất. Các database hỗ trợ horizontal scaling sẽ phân phối dữ liệu trên nhiều máy trong cluster.

Horizontal scaling thường được thực hiện thông qua database sharding hoặc data replication.

Database sharding là việc chia dữ liệu thành các phần nhỏ (shards) và phân phối chúng lên nhiều server khác nhau. Điều này giúp giảm tải cho từng server riêng lẻ.

Một số chiến lược sharding phổ biến bao gồm:

- Range-based sharding: phân phối dữ liệu dựa trên khoảng giá trị của một key.
- Directory-based sharding: sử dụng một dịch vụ tra cứu để điều hướng request đến database phù hợp.
- Geographical sharding: chia database dựa trên vị trí địa lý.

Một lựa chọn horizontal scaling khác là data replication. Đây là việc giữ nhiều bản sao của dữ liệu trên các server khác nhau để đảm bảo tính sẵn sàng cao (high availability).

Có hai mô hình replication phổ biến:

- Master–Slave replication: một database master xử lý ghi, và nhiều slave chỉ đọc.
- Master–Master replication: nhiều database có thể vừa đọc vừa ghi.

Việc scale database là một chuyện, nhưng bạn cũng cần truy cập dữ liệu nhanh hơn. Vì vậy, hãy nói về các kỹ thuật tối ưu hiệu năng database.

Kỹ thuật dễ thấy nhất là caching. Caching không chỉ dành cho web server — database cũng có thể cache. Việc cache database thường được thực hiện bằng in-memory database như Redis, giúp cache các truy vấn phổ biến và tăng hiệu năng đáng kể.

Kỹ thuật tiếp theo là indexing. Index là một cách khác để cải thiện hiệu năng database. Việc tạo index cho các cột thường xuyên được truy cập sẽ giảm đáng kể thời gian truy xuất dữ liệu.

Kỹ thuật tiếp theo là query optimization. Bạn có thể tối ưu các câu truy vấn để truy cập dữ liệu nhanh hơn, ví dụ như giảm số lượng join, và sử dụng các công cụ như SQL Query Analyzer hoặc EXPLAIN plan để hiểu rõ hiệu năng của query.

Trong mọi trường hợp, bạn nên nhớ đến CAP Theorem. Định lý CAP nói rằng một hệ thống chỉ có thể đảm bảo tối đa hai trong ba yếu tố sau:

- Consistency (tính nhất quán)
- Availability (tính sẵn sàng)
- Partition Tolerance (khả năng chịu phân mảnh mạng)

Khi thiết kế hệ thống, bạn cần ưu tiên hai yếu tố dựa trên yêu cầu của bài toán trong buổi phỏng vấn.